

Generative classifiers are just language models: some simple tricks

Jialin Lu 2026-06-08

A generative classifier is a language model fine-tuned to write down a label. It is a good way to build a classifier and a slow way to run one: the model runs full text generation to produce what is really a choice among a few labels. This post explains what such a model actually computes, and three small changes that remove the wasted work. None of the three is new; they are the ordinary tricks for serving a language model, and they apply here because a generative classifier is a language model. We measure on CPU, using Qwen3Guard-Gen-0.6B.

Contents

Two ways to build a classifier	1
What the model computes	2
Where the time goes	3
Three tricks	4
L1: stop decoding	4
L2: only the last position	5
L3: reuse the fixed prompt	5
Results	5
Summary	6
Appendix: GPU and the larger models	6
Bibliography	7

Two ways to build a classifier

Say we want to check text for safety, sorting each input into safe, unsafe, or controversial, and we want it cheap, because it runs on every message. A classifier does this: it takes an input and returns one of K labels, here $K = 3$. There are two ways to build one.

The standard way is a small head on top of an encoder. The encoder turns the input x into a vector $h(x) \in \mathbb{R}^d$, a linear layer of K units turns that into K scores, and a softmax turns the scores into a distribution over the labels,

$$p(y = k \mid x) = \frac{\exp(w_k^\top h(x))}{\sum_{j=1}^K \exp(w_j^\top h(x))}, \quad W \in \mathbb{R}^{K \times d}. \quad (1)$$

You train W and the encoder on labelled data with cross-entropy, and that is the whole model. This is a BERT-style text classifier. It is small, fast, and for many tasks the right choice.¹

The second way, common lately for content safety, is to take a pretrained, instruction-tuned language model and fine-tune it to write the label as text. Llama

¹ Meta Llama Team. 2024. "Llama-Prompt-Guard-2-86m Model Card."

Guard², ShieldGemma³, and Qwen3Guard⁴ all do this. You give the model a chat prompt and it generates

Safety: Unsafe
Categories: Violent

and you read the word after `Safety:`. I will call this a *generative classifier*, to set it apart from the head above.

Running a few-hundred-million-parameter model to choose among three labels is a lot of computation. But the generative version gives you things the small head cannot. A head trained from scratch only knows the labels it was shown, and it tends to do poorly on inputs that do not resemble its training data. The pretrained model already knows a great deal about language and the world, in many languages; a head trained on your labels does not. It also lets you keep the policy in the prompt: the list of what counts as unsafe is part of the system prompt, not the weights, so you change it by editing text instead of retraining.⁵ So the generative classifier is often the right tradeoff. As usually run, it is just not fast. The rest of this post makes it fast, and the first step is to be exact about what it computes.

² Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

³ Zeng, Wenjun, and others. 2024. "Shield-gemma: Generative AI Content Moderation Based on Gemma."

⁴ Zhao, Haiyang, and others. 2025. "Qwen3guard Technical Report."

Listing 1: The model card's output format. The label is the word after `Safety:`.

⁵ Alibaba ship a strict and a loose Qwen3Guard with the same weights and different system wording. A from-scratch classifier cannot do that; changing its label set means training a new one.

What the model computes

A language model does not have the head from Equation 1. It has a *language-modeling head*, the same kind of linear layer as W above, only much wider: at each position t it projects the hidden state onto one score per vocabulary token, giving a distribution over the whole vocabulary $\mathcal{V}$ for the next token,

$$p(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(E_v^\top h_t)}{\sum_{u \in \mathcal{V}} \exp(E_u^\top h_t)}, \quad E \in \mathbb{R}^{|\mathcal{V}| \times d}. \quad (2)$$

Here h_t is the hidden state at position t , and E is that wide projection, one row per vocabulary token. The vocabulary is large; for Qwen3 it is about 150,000 tokens. That is the wrong shape for a classifier: a distribution over the vocabulary, not over $K = 3$ labels.

To get a classifier out of it, make the K labels themselves be K tokens. `safe`, `unsafe`, and `controversial` are each a single token; call their ids i_1, i_2, i_3 . End the prompt so the next token the model would write is the verdict, by ending it with `Safety:` , which we come back to below. Then read Equation 2 at that one position and keep only those three logits, the raw pre-softmax scores:

$$p(y = k \mid x) = \frac{\exp(E_{i_k}^\top h_t)}{\sum_{j=1}^K \exp(E_{i_j}^\top h_t)}. \quad (3)$$

This is Equation 2 restricted to the label tokens and renormalized. Put Equation 3 next to Equation 1 and they are the same computation. The classification head W is already present, as the three rows $E_{i_1}, E_{i_2}, E_{i_3}$ of the language-modeling

head. We do not add a head; we read three rows of the head already there, at one position.

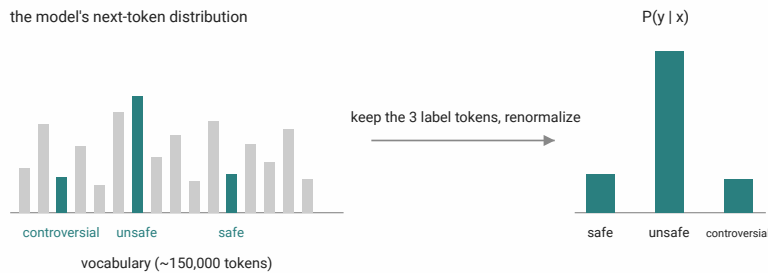


Figure 1: Classification as a distribution: keep the three label tokens out of the next-token distribution over the whole vocabulary, and renormalize. The classification head is three rows of the language-modeling head.

The chat template and the literal `Safety:` are there only to put the model in the state where its next token is the verdict. They are not part of the answer. The rest of the post depends on one fact:

Classification reads the next-token distribution at one position, over K tokens, after a fixed prompt.

Anything the model does beyond that read does not affect the label.

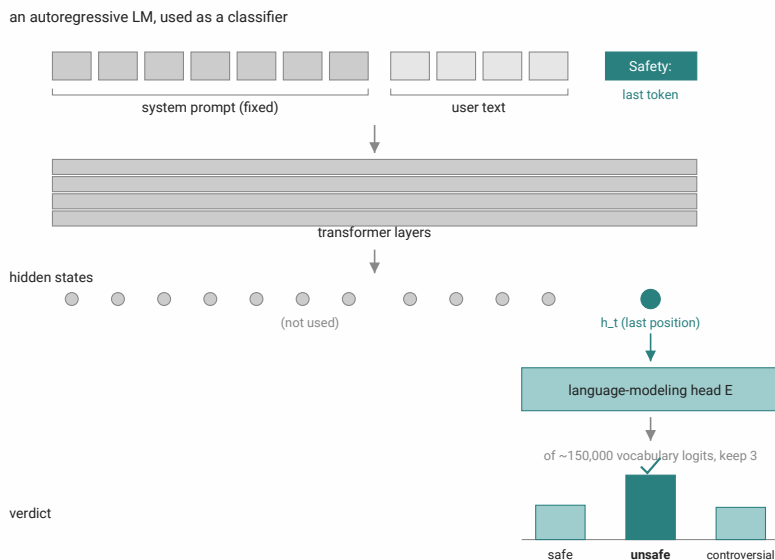


Figure 2: An autoregressive language model used as a classifier. The whole prompt runs through the model, but only the last position's hidden state is read, and only the three label rows of the head matter.

Where the time goes

The model-card recipe renders the chat prompt, calls `generate()` for a handful of tokens, and parses the text. One call is therefore a *prefill* pass that encodes

the N prompt tokens, followed by a *decode loop* that produces the output one token at a time; each token is another full forward pass.

Qwen3Guard writes about nine tokens before the line we want is complete,⁶ so the decode loop runs about nine times. Roughly, we pay for ten forward passes, one prefill plus nine decode, when we need one. On a small model a forward pass is cheap in arithmetic but heavy in fixed overhead, and nine in sequence are most of the wall-clock. On a CPU that is the difference between two seconds and a fifth of a second.

⁶ Safety: Unsafe, then the Categories: line it was fine-tuned to always produce.

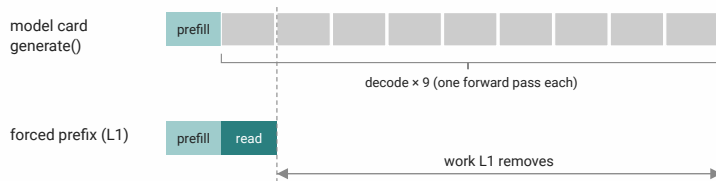


Figure 3: One call under the model-card recipe: a prefill pass and nine decode steps. The label is fixed by the end of the first step, so the rest is unused.

Compare that against the fact above. We need the distribution at one position; the decode loop computes nine. We need three logits; Equation 2 projects every position onto all 150,000 tokens. We send the same fixed prompt every call; the model re-encodes it from scratch each time. Three pieces of unused work, and three tricks that remove them.

Three tricks

Forced decoding, reading the logits at the last position, and caching a fixed prompt's KV are ordinary parts of serving a language model. A generative classifier is a language model being served, so all three apply to it without modification. If a team treats the safety model as a black box, the way they would treat an older classifier, they pay for the full generation when they do not have to. The three tricks stack; each removes one of the three pieces of unused work above.

L1: stop decoding

Classification reads one position, so generating the rest of the string is unnecessary.

The model was fine-tuned to always begin its answer with Safety: , so forcing that prefix matches what it would generate anyway; we skip ahead to the token that varies. We supply the prefix ourselves: append the string Safety: to the prompt, treat it as input, run one forward pass, and read the next-token distribution at the last position. The argmax over the three label logits is the verdict. This is forced decoding: we feed the model the tokens it would have produced, instead of letting it generate them. ShieldGemma's model card describes the same procedure,⁷ and Llama Guard describes the first-token-logit version of it.⁸ There is no decode loop: one forward pass instead of about ten. This is the largest of the three savings, because it removes nine of the ten forward passes.

⁷ Zeng, Wenjun, and others. 2024. "Shield-gemma: Generative AI Content Moderation Based on Gemma."

⁸ Meta Llama Team. 2024. "Llama Guard 3-8b Model Card."

L2: only the last position

We read one position, so projecting the others onto the vocabulary is unnecessary.

The language-modeling head in Equation 2 is a matrix multiply of shape $[N, d] \times [d, |\mathcal{V}|]$: for each of the N positions, project a d -vector onto 150,000 logits. Equation 3 reads only the last position, so we slice the hidden states from $[N, d]$ to $[1, d]$ before the projection and compute that one row. The other $N - 1$ projections do not happen.⁹ The cost L2 removes is running that projection on every prefill position instead of only the last. Narrowing the projection from the full vocabulary to the three label rows is a separate idea, and on this model it saves under a millisecond, so we leave the head whole and slice only the position.

⁹ In PyTorch this is `logits_to_keep=1`; in ONNX it is a slice node on the graph; llama.cpp and most runtimes return only the last position already.

L3: reuse the fixed prompt

The system prompt, the chat template plus the policy text, is the same on every call, so we can reuse its work.

As the model reads the prompt, each token produces key and value vectors that later positions attend to; together these are the KV cache. The prompt prefix never changes, so its keys and values never change across calls, and we can compute them once and keep them.¹⁰ After the first call, each call pays only for the user's text, not for re-reading the policy.

¹⁰ In practice: cache the longest prefix that is the same regardless of user content, and run the forward pass only over the variable suffix.

Results

The three tricks change which numbers the model computes, not how each number is stored, so they apply on any backend and in any precision. We measure on CPU rather than GPU because, for a sub-billion-parameter model at batch one, that is the realistic deployment, and because the per-call savings are largest where each forward pass is expensive. The model is Qwen3Guard-Gen-0.6B, batch one, on sixteen cores of a Kunpeng 920¹¹ server CPU, across three backends: PyTorch, ONNX Runtime,¹² and llama.cpp.¹³ The input is a representative few hundred tokens, and each number is the median of 100 timed calls.

¹¹ Huawei. n.d. "Kunpeng 920 Processor"

¹² Microsoft. 2018. "ONNX Runtime."

¹³ Gerganov, Georgi, and the llama.cpp contributors. 2023. "Llama.cpp."

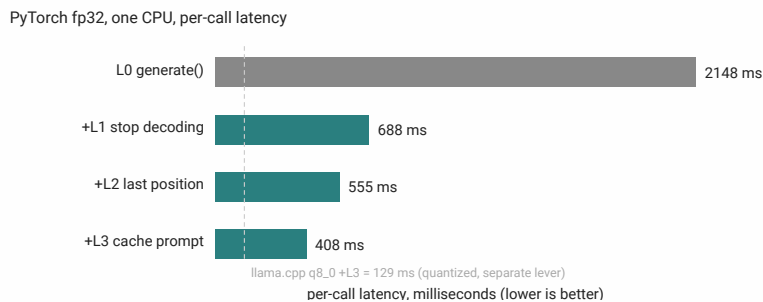


Figure 4: The ladder on one backend (PyTorch fp32). The decode loop, removed by L1, is most of the cost; L2 and L3 take the rest. Quantization and a tuned kernel go further, but are separate from the three tricks.

Table 1: Per-call latency, p50 ms, walking the ladder on three backends. L0 is the model-card `generate()` path. llama.cpp returns only the last position by default, so L2 is already part of its baseline; that row is also 8-bit quantized, which is why it starts lower.

backend	L0 baseline	+L1	+L2	+L3
PyTorch fp32	2148	688	555	408
ONNX fp32	1671	598	485	253
llama.cpp q8_0	643	261	–	129

The pattern is the same on each backend: L1 removes most of the time, and L2 and L3 take more off. The three tricks alone bring PyTorch from 2148 to 408 ms, about five times faster, and the model's output does not change.¹⁴

Two more changes help, but are not among the three tricks. Quantizing the weights to 8 bits (the llama.cpp q8_0 row) costs almost nothing in accuracy here and brings the best setup to 129 ms. A BLAS kernel matched to the CPU trims the fp32 path further. Both add to the tricks rather than replace them.

¹⁴ We check this on every sample: the fast path returns the same verdict as the model-card path, and the fp32 logits match.

Summary

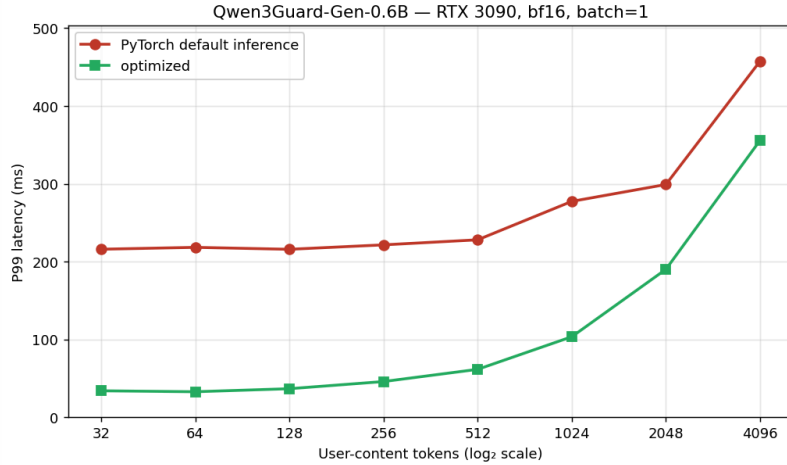
A generative classifier is a language model asked to write down a label. Written out, classification is the model's next-token distribution, read at one position, over K tokens, after a fixed prompt. Its classification head is three rows of the model's own vocabulary projection. Everything past that read is unused, and the three tricks remove it: stop decoding, read only the last position, reuse the fixed prompt. They apply on any backend, in any precision, and leave the output unchanged. Together they take a two-second CPU call down to about four hundred milliseconds, before any quantization.

The tricks are standard. A generative classifier is a language model, so you get them as soon as you stop treating it as a black box. The same holds whenever a generative model produces a short, fixed-shape answer: most of the generation is not part of the answer.

Appendix: GPU and the larger models

On a GPU the forced-prefix trick (L1) is still the one that matters, because the decode loop's fixed per-step cost dominates there too. On an RTX 3090, Qwen3Guard-Gen-0.6B with L1 runs about 29 ms p50, against 237 ms for the model-card path, at a representative input length. L2 and L3 do not help on the GPU, where the vocabulary projection and prompt re-encoding are cheap next to per-call overhead. On CPU the opposite holds, which is why the full ladder matters there.

Figure 5: Qwen3Guard-Gen-0.6B on an RTX 3090: the model-card path against the forced-prefix path (L1), p99 latency across input lengths. The forced-prefix path stays under a 200 ms budget out to about 2048 input tokens.



The same construction works on the larger members of the family.

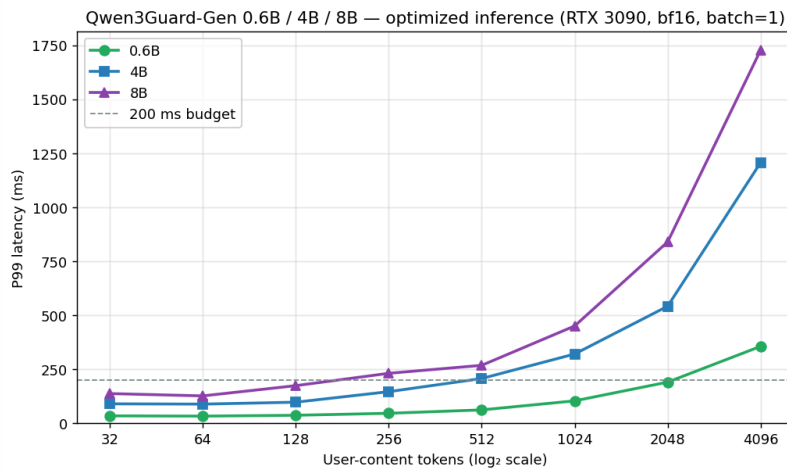


Figure 6: The forced-prefix path across the three Qwen3Guard-Gen sizes (0.6B / 4B / 8B) on an RTX 3090. All three get the same trick; the 0.6B clears a 200 ms budget at the longest inputs, the larger two hit it sooner.

Bibliography

- Gerganov, Georgi, and the Llama.cpp contributors. 2023. “Llama.cpp.”
- Huawei. n.d. “Kunpeng 920 Processor.”
- Meta Llama Team. 2024. “Llama Guard 3-8b Model Card.”
- Meta Llama Team. 2024. “Llama-Prompt-Guard-2-86m Model Card.”
- Microsoft. 2018. “ONNX Runtime.”
- Zeng, Wenjun, and others. 2024. “Shieldgemma: Generative AI Content Moderation Based on Gemma.”
- Zhao, Haiyang, and others. 2025. “Qwen3guard Technical Report.”